RESEARCH masque

RESEARCH masque

Revision: 1 **Last modified:** 2026-07-04T12:00:00Z

Web access: YES. Researched 2026-06-25. Topic: MASQUE (RFC 9298 CONNECT-UDP / 9297 HTTP Datagrams / 9221 QUIC DATAGRAM) for tunnelling WireGuard over HTTP/3, Rust + Go ecosystem maturity, and how Mullvad ships WG-over-MASQUE QUIC obfuscation. For a Mullvad-parity self-hosted VPN spec.

---

1. The RFC stack (the standardised building blocks)

WG-over-MASQUE is a layering of four IETF RFCs. All are published Proposed Standards (stable, not drafts):

- **RFC 9298 — Proxying UDP in HTTP (CONNECT-UDP).** The MASQUE control plane. An HTTP client opens a tunnel for UDP through an HTTP proxy, analogous to `CONNECT` for TCP. Over HTTP/3 the client issues an **Extended CONNECT** (`:protocol = connect-udp`) request to a URI template like `https://proxy.example.org/.well-known/masque/udp/{target_host}/{target_port}/`. Proxy opens a UDP socket to the target and relays. [9298]
- **RFC 9297 — HTTP Datagrams and the Capsule Protocol.** Defines the `HTTP Datagram` abstraction + Quarter Stream ID multiplexing that ties each datagram to its CONNECT-UDP request stream. Over HTTP/3, HTTP Datagrams ride the QUIC DATAGRAM frame (unreliable); over HTTP/2 / fallback they ride the reliable Capsule Protocol on the stream. [9297]
- **RFC 9221 — QUIC DATAGRAM frame extension.** Unreliable, un-retransmitted datagram frame inside a QUIC connection. This is what makes UDP-over-QUIC efficient (no head-of-line blocking, no retransmit of stale WG packets — matching WireGuard's own loss-tolerant UDP semantics). Negotiated via the `max_datagram_frame_size` transport parameter. [9221]

- **RFC 9220 — Bootstrapping WebSockets / Extended CONNECT with HTTP/3.** Adds the `:protocol` pseudo-header (Extended CONNECT) that CONNECT-UDP requires over HTTP/3. [via masque-go docs]

Data-plane flow: client encapsulates one WireGuard UDP packet as one HTTP Datagram (RFC 9297) → carried in a QUIC DATAGRAM frame (RFC 9221) tagged with the CONNECT-UDP stream's Quarter Stream ID → proxy (RFC 9298) extracts the payload and emits a plain UDP packet to the WireGuard server; reverse for return traffic. [9298, 9297, 9221]

Related-but-distinct: **RFC 9484 — Proxying IP in HTTP (CONNECT-IP)** is the full-tunnel sibling (proxies IP packets, not just UDP). Not what Mullvad uses for WG obfuscation, but relevant if Helix later wants a full L3 MASQUE tunnel.

---

2. How Mullvad implements it (the parity target)

Mullvad shipped **QUIC obfuscation for WireGuard** on desktop in **2025.9** (Sept 2025), and on Android & iOS in app version **2025.8+** (rolled out later in 2025). [mullvad-blog, cyberinsider]

Architecture (confirmed facts):

- Built directly on **MASQUE / RFC 9298**: WireGuard's UDP is tunnelled through an HTTP/3 proxy so the wire traffic "appears as web traffic" (HTTP/3 over QUIC on UDP/443). [mullvad-blog]
- Mullvad runs the **proxy server-side on its relays** (the in- address); a 2025.12/2026.x fix notes the client now randomly selects one of several available in-addresses per connection attempt. [mullvad-changelog]
- **Implementation language = Rust**, inside the `mullvad-daemon` crate graph. Strong evidence the QUIC stack is **quinn**: Mullvad changelog explicitly references the **`quinn_udp`** crate ("`quinn_udp` was flooding mullvad-daemon.log with warnings"). quinn_udp is the UDP I/O crate of the `quinn-rs/quinn` Rust QUIC implementation. [mullvad-changelog, quinn]
- **Default behaviour:** auto-tries QUIC after a few failed normal/ obfuscation connection attempts; user can force it (`mullvad obfuscation set mode quic`). Intended for restrictive networks that allow web traffic only. [mullvad-blog]
- **Throughput cost is real and acknowledged:** "wraps the WireGuard tunnel in a QUIC tunnel … computationally very expensive and affects throughput" — use only when needed. This is the double-encryption + double-congestion-control tax

(WG crypto inside QUIC crypto, two AEAD layers, QUIC CC over UDP). [mullvad-blog]

DPI resistance / "looks like a browser" (what is public vs unstated):

- Claimed mechanism = **collateral-damage deterrence**: traffic is HTTP/3 on QUIC/UDP 443; blocking it risks breaking the open web (Google, YouTube, Cloudflare all use HTTP/3). Censors face high false-positive cost. [cyberinsider, mullvad-blog]
- **NOT publicly documented:** exact TLS/QUIC ClientHello fingerprint matching (uTLS-style), SNI value used, ALPN, domain-fronting, or how closely the QUIC Initial mimics Chrome/Firefox. Honest gap — Mullvad's blog is deliberately vague on the anti-fingerprinting layer. A serious parity effort must treat QUIC/TLS fingerprint mimicry as an *open design problem*, not a solved one.

---

3. Rust ecosystem maturity — turnkey vs hand-rolled (KEY FINDING)

There is NO turnkey Rust crate that gives you RFC 9298 CONNECT-UDP client+proxy out of the box. You hand-roll it on top of quinn + h3. This is exactly what Mullvad did.

Building blocks available:

- **`quinn` (quinn-rs/quinn)** — mature, production async Rust QUIC (IETF QUIC, rustls-based TLS 1.3). `quinn-proto` = sans-I/O state machine. Supports the QUIC DATAGRAM extension (RFC 9221). This is the solid foundation. [quinn]
- **`h3` (hyperium/h3)** — HTTP/3, generic over the QUIC transport (pairs with quinn via `h3-quinn`). Mature-ish but the pieces CONNECT-UDP needs are **experimental and split into separate crates**:
  - **`h3-datagram` (v0.0.2)** — RFC 9297 HTTP Datagram support. Pre-1.0, "API subject to change." [h3-releases]
  - **Extended CONNECT** — added as a separate modular feature (needed for the `:protocol` pseudo-header). [h3-discussion-189]
  - **`h3-webtransport`** — "API subject to change … may contain bugs … not yet complete." (WebTransport is the adjacent extended-CONNECT use case; shows the maturity level of this corner of h3.) [h3-webtransport]
- **No published `masque/connect-udp` crate** wires these into an RFC 9298 client+proxy. Third-party experiments exist (e.g. `jromwu/masquerade` — "an implementation of MASQUE in Rust") but are research-grade, not maintained libraries. [masquerade]

- **`tokio-quiche` (Cloudflare, open-sourced Dec 2025)** — async QUIC + HTTP/3 Rust library over Cloudflare's `quiche`. Newer alternative QUIC base; still no turnkey CONNECT-UDP helper. [infoq-tokio-quiche]

Verdict (Rust): hand-roll CONNECT-UDP on `quinn` (+ optionally `h3` / `h3-datagram` for the Extended-CONNECT handshake), using QUIC DATAGRAM frames for the data plane. Expect to implement: the Extended-CONNECT request/response, Quarter-Stream-ID datagram mux, and the proxy UDP-socket relay yourself. Mullvad's quinn_udp dependency confirms this is the trodden path. Budget it as real engineering, not a dependency add.

---

4. Go ecosystem maturity — there IS a turnkey library

- **`masque-go` (quic-go/masque-go)** — turnkey RFC 9298 CONNECT-UDP, on top of `quic-go`. Provides **both client and proxy**. Current release **v0.3.0 (2025-06-24)**, MIT, actively maintained (tracks latest two Go releases). API: client `Dial` methods returning a proxied UDP conn; a `Proxy` struct implementing the RFC 9298 proxy; uses HTTP Datagrams (RFC 9297) over quic-go's DATAGRAM support. Maturity: "specialised, active" — pre-1.0, ~236★, basic CONNECT-UDP; CONNECT-IP not advertised. [masque-go, masque-go-pkg]
- **`quic-go`** itself — mature production Go QUIC/HTTP/3 (used by Caddy, etc.), supports HTTP Datagrams (RFC 9221/9297). CONNECT-UDP lives in masque-go, not core. [quic-go-datagrams]

Verdict (Go): masque-go is the closest thing to a drop-in WG-over-MASQUE substrate. You still write the WireGuard-packet ↔ proxied-UDP-conn glue and the client-side dialer/listener, but the RFC 9298 protocol machinery is done. Trade-off vs Rust: less control over the QUIC/TLS fingerprint (harder to do Chrome-mimicry / uTLS-equivalent in Go), which is the very thing DPI resistance hinges on.

---

5. Implications for a Mullvad-parity Helix VPN spec

- **Protocol choice:** WireGuard data plane unchanged; add a MASQUE (RFC 9298 CONNECT-UDP over HTTP/3) obfuscation transport that encapsulates the WG UDP socket. Use QUIC DATAGRAM (RFC 9221) for the data plane — never

the reliable stream — to preserve WG's loss-tolerant semantics and avoid HoL blocking.
- **Language trade-off:**
  - Rust + quinn = max control (TLS/QUIC fingerprint, perf, matches Mullvad) but you hand-roll RFC 9298. Highest parity fidelity.
  - Go + masque-go = fastest to a working tunnel (turnkey RFC 9298) but weaker fingerprint control. Good for a first iteration / reference proxy.
- **Server side:** a self-hosted HTTP/3 proxy terminating CONNECT-UDP on UDP/443 and relaying to the local WireGuard endpoint. Both quinn and quic-go/masque-go can host this.
- **Performance budget (anti-bluff):** plan for measurable throughput loss (double crypto + double congestion control). Spec MUST require captured throughput evidence WG-direct vs WG-over-MASQUE; do NOT claim parity speed.
- **DPI/fingerprint = the hard, unsolved part:** "looks like HTTP/3" via collateral-damage is the public Mullvad story, but real DPI resistance needs QUIC Initial / TLS ClientHello mimicry of a real browser (ALPN `h3`, realistic SNI, Chrome-like transport params). Mullvad does NOT publish this layer — treat it as an explicit research/risk item in the spec, not a checkbox.
- **Reuse before reimplement (§11.4.74):** Go path → adopt `quic-go/masque-go` upstream; Rust path → adopt `quinn` + `h3`/`h3-datagram` and contribute any CONNECT-UDP helper upstream rather than forking.

Sources verified

- RFC 9298 Proxying UDP in HTTP — https://datatracker.ietf.org/doc/rfc9298/ (accessed 2026-06-25)
- RFC 9297 HTTP Datagrams and the Capsule Protocol — https://datatracker.ietf.org/doc/html/rfc9297 (accessed 2026-06-25)
- RFC 9221 QUIC DATAGRAM (referenced via search result summaries) — https://datatracker.ietf.org/doc/rfc9298/ + https://quic-go.net/docs/http3/datagrams/ (accessed 2026-06-25)
- Mullvad blog — Introducing QUIC Obfuscation for WireGuard — https://mullvad.net/en/blog/introducing-quic-obfuscation-for-wireguard (accessed 2026-06-25)
- Mullvad blog — QUIC Obfuscation now available on Android and iOS — https://mullvad.net/en/blog/quic-obfuscation-now-available-on-android-and-ios (accessed 2026-06-25)
- CyberInsider — Mullvad Adds QUIC Obfuscation for WireGuard — https://cyberinsider.com/mullvad-adds-quic-obfuscation-for-wireguard-to-evade-censorship/ (accessed 2026-06-25)

- mullvadvpn-app CHANGELOG (quinn_udp ref, in-address selection, QUIC fixes) — https://github.com/mullvad/mullvadvpn-app/blob/main/CHANGELOG.md (accessed 2026-06-25)
- mullvadvpn-app repo — https://github.com/mullvad/mullvadvpn-app (accessed 2026-06-25)
- quinn (quinn-rs/quinn) — https://github.com/quinn-rs/quinn + https://docs.rs/quinn/latest/quinn/ (accessed 2026-06-25)
- h3 (hyperium/h3) releases + WebTransport/datagram discussion — https://github.com/hyperium/h3/releases + https://github.com/hyperium/h3/discussions/189 (accessed 2026-06-25)
- h3-webtransport (maturity note) — https://lib.rs/crates/h3-webtransport (accessed 2026-06-25)
- masque-go (quic-go/masque-go) — https://github.com/quic-go/masque-go + https://pkg.go.dev/github.com/quic-go/masque-go (accessed 2026-06-25)
- masque-go releases (v0.3.0, 2025-06-24) — https://github.com/quic-go/masque-go/releases (accessed 2026-06-25)
- quic-go CONNECT-UDP docs — https://quic-go.net/docs/connect-udp/ (accessed 2026-06-25)
- quic-go HTTP Datagrams docs — https://quic-go.net/docs/http3/datagrams/ (accessed 2026-06-25)
- jromwu/masquerade (Rust MASQUE, research-grade) — https://github.com/jromwu/masquerade (accessed 2026-06-25)
- InfoQ — Cloudflare open-sources tokio-quiche — https://www.infoq.com/news/2025/12/quic-http3-rust/ (accessed 2026-06-25)